

Національний університет "Львівська політехніка"

Інститут комп'ютерних наук та інформаційних технологій

Кафедра систем штучного інтелекту

Спеціальність 122 «Комп'ютерні науки»



ARTIFICIAL
INTELLIGENCE

ПОЯСНЮВАЛЬНА ЗАПИСКА

до бакалаврської кваліфікаційної роботи на тему:

«Система для автоматичної нормалізації суржику та діалектних форм української мови»

Студента(ки) групи

ІІІ-42

Дениса КОВАЛЯ

Керівник роботи

(підпис)

(Олексій СИВОКОНЬ)

Консультанти

(підпис)

(_____)

(підпис)

(_____)

Завідувач

кафедри СШІ

(Наталія МЕЛЬНИКОВА)

Львів - 2025

Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій
Кафедра систем штучного інтелекту
Спеціальність 122 «Комп’ютерні науки»

ЗАТВЕРДЖУЮ:

Завідувач кафедри СШІ _____

“ _____ ” _____ 2025р.

ЗАВДАННЯ
НА БАКАЛАВРСЬКУ КВАЛІФІКАЦІЙНУ РОБОТУ СТУДЕНТОВІ

Ковалю Денису Петровичу

1. Тема роботи: «Система для автоматичної нормалізації суржику та діалектних форм української мови» затверджена наказом університету № 902-4-08 від 12.03.2025

2. Термін подання закінченої роботи: _06.06.2025

3. Вхідні дані до (проекту) роботи: літературні джерела щодо методів машинного перекладу, словники суржиків та діалектів, методичні рекомендації до виконання бакалаврської кваліфікаційної роботи для студентів спеціальності 122 «Комп’ютерні науки».

4. Зміст розрахунково-пояснювальної записки: (перелік питань, що належить розробити): аналітичний огляд літературних та інших джерел, системний аналіз та обґрунтування проблеми, методи та засоби вирішення проблеми, практична реалізація.

5. Перелік графічного матеріалу (з точним зазначенням обов’язкових креслень):
діаграма класів

6. Консультанти по роботі, із зазначенням розділів проекту, що стосується їх

Розділ	Консультант	Підпис, дата	
		Завдання видав	Завдання прийняв
Консультант з іноземної мови			

7. Дата видачі завдання _____ 16.09.2024 _____

Керівник роботи _____
(підпис)

Завдання прийняв до виконання _____
(підпис)

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів дипломної роботи	Термін виконання етапів роботи	Примітка
1	Огляд літератури	16.09.2024 – 28.10.2024	виконано
2	Створення архітектури	29.10.2024 – 16.12.2024	виконано
3	Розробка алгоритмічного та програмного забезпечення	17.12.2024 – 10.03.2025	виконано
4	Експериментальні дослідження	11.03.2025 – 29.04.2025	виконано
5	Оформлення записки	30.04.2025 – 06.06.2025	виконано

Керівник роботи _____
(підпис)

Студент _____
(підпис)

АНОТАЦІЯ

Бакалаврська кваліфікаційна робота виконана студентом групи КН-412 Шиманським Володимиром Михайловичем. Тема “Тема роботи”. Робота направлена на здобуття ступеня бакалавра за спеціальністю 122 «Комп’ютерні науки».

Об’єктом дослідження є процеси підвищення точності розв’язання задач класифікації.

Предметом досліджень є методи та алгоритми побудови гібридних систем обчислювального інтелекту.

Досягнення мети відбувається за рахунок розширення вимірності простору вхідних даних задачі шляхом використання виходів ймовірнісної нейронної мережі. Подальше опрацювання розширеного набору даних відбувається із використанням відомих методів машинного навчання. Апробацію роботи гібридної системи здійснено з використанням різних алгоритмів машинного навчання на основі реального біомедичного набору даних для розв’язання задачі класифікації.

У результаті виконання дипломної роботи створено гібридну систему обчислювального інтелекту; розроблено та запатентовано її програмну реалізацію, яка дозволяє досліджувати роботу різних методів машинного навчання, підбирати гіперпараметри для оптимальної роботи системи, а також порівнювати результати навчання класифікаторів на трьох різних алгоритмах розширених початкових даних.

Загальний обсяг роботи: 70 сторінок, 33 рисунки, 34 посилання.

Ключові слова: ймовірнісна нейронна мережа, класифікація, розширення вибірки, машинне навчання.

ABSTRACT

The bachelor's qualification work was completed by a student of the KN-412 group Shymanskyi Volodymyr Mychailovych. The topic is "Topic name". The work is aimed at obtaining a bachelor's degree in 122 "Computer Science".

The object of research is the process of increasing the accuracy of solving classification problems.

The subject of research is methods and algorithms for building hybrid systems of computational intelligence.

The goal of the research is achieved by expanding the dimensionality of the input data space of the problem by using the outputs of the probabilistic neural network. Further processing of the extended data set is carried out by using known methods of machine learning. The performance of the hybrid system was tested by using various machine learning algorithms based to solve the classification problem on a real biomedical data set.

As a result of the bachelor's qualification work, a hybrid system of computing intelligence was created; developed and patented its software implementation, which allows you to study the performance of various machine learning methods, select hyperparameters for optimal system operation and also compare the results of training classifiers on three different algorithms of extended initial data.

The total volume of work: 70 pages, 33 figures, 34 references.

Keywords: probabilistic neural network, classification, sample expansion, machine learning.

ЗМІСТ

АНОТАЦІЯ	4
ABSTRACT	5
ЗМІСТ	6
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	7
ВСТУП	13
РОЗДІЛ 1. ЗАГАЛЬНІ ПОЛОЖЕННЯ	16
1.1. Аналіз літературних джерел.....	16
1.2. Аналіз наборів даних.....	18
1.3. Висновки до розділу.....	25
РОЗДІЛ 2. ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ	27
2.1. Аналіз методів і підходів до нормалізації діалектів та суржику.....	27
2.2. Постановка задачі та інформаційна модель даних.....	29
2.3. Математичні основи використаних моделей.....	31
2.4. Вибір моделі та обґрунтування: від бейзлайну до Lora LLM.....	32
2.5. Метрики якості та цільові функції навчання.....	36
2.6. Інформаційне забезпечення навчання: баланс, семплінг, контроль доменів.....	38
2.7. Висновки до розділу.....	39
РОЗДІЛ 3. ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ	40
3.1. Засоби реалізації.....	40
3.2. Вимоги до технічного та програмного забезпечення.....	40
3.3. Висновки до розділу.....	40
ЗАГАЛЬНІ ВИСНОВКИ	41
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	42
Додаток А.	44
Додаток Б.	45

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API - Application Programming Interface, програмний інтерфейс (інтерфейс програмування застосунків, інтерфейс прикладного програмування) — набір визначень підпрограм, протоколів взаємодії та засобів для створення програмного забезпечення.

AI - Artificial Intelligence — популярне англomовне скорочення для позначення «Штучного інтелекту».

ML – Machine learning (Машинне Навчання)

DL – Deep learning (Глибоке Навчання)

UMAP – Uniform Manifold Approximation

t-SNE – t-distributed stochastic neighbor embedding

CART - Classification and Regression Tree

Структура бакалаврської кваліфікаційної роботи

Таблиця 1.1. Опис стилів, які потрібно використовувати при
форматуванні роботи

Назва розділу	Кількість сторінок (включно з підрозділами)
Титульний аркуш	1
Завдання на бакалаврську кваліфікаційну роботу	2
Анотація	1
Abstract	Якщо захист <i>українською мовою</i> – 1 ст. Якщо захист <i>іноземною мовою</i> – 10-15 ст.
Зміст	1-2
Вступ	2-5
Розділ 1. Загальні положення	10-15
Розділ 2. Інформаційне та математичне забезпечення	15-25
Розділ 3. Програмне та технічне забезпечення	15-25
Загальні висновки	1-2
Список використаних джерел (15-30 посилань)	1-2
Додатки (за наявності)	0-50
Загальний обсяг роботи	50-95 (+додатки)

Короткий опис стилів та інструкцій:

1. На сторінці 1-3 потрібно вести відповідну інформацію у тих місцях, які позначені жовтим кольором. При цьому змінити тип зафарбовування тексту на «Без кольору»
2. Зміст генерується автоматично на основі стилів Header_Level_1, Header_Level_2 та Header_Level_3.
3. У вступі обов'язково мають бути абзаци, що позначені жирним шрифтом.
4. Формули оформлювати за зразком наведеним на сторінці 13.
5. Список використаних джерел має бути відсортований по алфавіту.

У тексті записки обов'язково повинні бути посилання на усі джерела, що будуть вказані у розділі «Список використаних джерел» [2]. Посилання здійснювати за допомогою інструменту «Перехресні посилання».

6. Таблиця стилів наведено у Таблиці 1.1. Для форматування записки використовувати тільки зазначені стилі.

Таблиця 1.2. Опис стилів, які потрібно використовувати при форматуванні роботи

Назва стилю	Застосування стилю
Header_Level_1	Використовується для форматування назв розділів. Наприклад: Вступ, Розділ 1 і т.п. <i>Опис:</i> <i>Шрифт: Times New Roman, 16 п, напівжирний</i> <i>Відступ:</i> <i>Перший рядок: 0 см, По центру,</i> <i>Інтервал:</i> <i>Перед: 20 пт</i> <i>Після: 6 пт</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
Header_Level_2	Використовується для форматування назв підрозділів. Наприклад: 1.2. Сучасний стан досліджень предметної області. <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п, напівжирний</i> <i>Відступ:</i> <i>Перший рядок: 1,27 см, За шириною</i> <i>Інтервал:</i>

	<i>Перед: 6 пт</i> <i>Після: 6 пт</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
Header_Level_3	Використовується для форматування назв підрозділів нижчого рівня. Наприклад: 1.2.1. Загальні властивості рукописного тексту. <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п, напівжирний, курсив</i> <i>Відступ:</i> <i>Перший рядок: 1,27 см, За шириною</i> <i>Інтервал:</i> <i>Перед: 6 пт</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
Main_Text	Основний текст <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п,</i> <i>Відступ:</i> <i>Перший рядок: 1,27 см, За шириною</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
Marked_List	Маркований список <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п,</i> <i>Відступ:</i> <i>Зліва: 1,27 см</i> <i>Нависаючий відступ: 1,22 см, маркований +</i> <i>Рівень: 1 + Вирівняти за: 1,27 см + Відступ: 2,49 см</i> <i>Міжрядковий інтервал: 1,5 рядка</i>

Numbered_List	Нумерований список <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п,</i> <i>Відступ:</i> <i>Зліва: 1,37 см</i> <i>Нависаючий відступ: 0,63 см, нумерований +</i> <i>Рівень: 1 + Стиль нумерації: 1, 2, 3, ... +</i> <i>Почати з: 1 + Вирівнювання: зліва +</i> <i>Вирівняти за: 1,9 см + Відступ: 2,54 см</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
Figure_Title	Підпис малюнку <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п, напівжирний, курсив</i> <i>Відступ:</i> <i>Перший рядок: 0 см, По центру</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
Table_Title	Назва таблиці <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п, напівжирний</i> <i>Відступ:</i> <i>Перший рядок: 0 см, Справа</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
References	Список використаних джерел <i>Опис:</i> <i>Шрифт: Times New Roman, 14 п</i> <i>Відступ:</i>

	<i>Зліва: 0,5 см</i> <i>Нависаючий відступ: 0,75 см, нумерований +</i> <i>Рівень: 1 + Стиль нумерації: 1, 2, 3, ... +</i> <i>Почати з: 1 + Вирівнювання: зліва +</i> <i>Вирівняти за: 1,27 см + Відступ: 2,49 см</i> <i>Міжрядковий інтервал: 1,5 рядка</i>
--	---

ВСТУП

Актуальність теми зумовлена високою варіативністю української мови в реальній комунікації: поряд із літературною нормою поширені регіональні діалекти та російсько-український суржик. В Україні внутрішня міграція була доволі поширеною і раніше, а після початку повномасштабної війни стала ще помітнішою: люди частіше опиняються в нових мовних середовищах, де діалектні слова, суржик або локальні форми можуть бути незрозумілими у побуті, навчанні чи роботі. Подібні труднощі виникають і в абітурієнтів та студентів із сіл, які, переїжджаючи до міст, зберігають звичну лексику. За таких умов інструмент «діалект/суржик → літературна українська» може слугувати простим засобом нормалізації коротких текстів — повідомлень, оголошень, чатів.

Незважаючи на прогрес у задачах обробки природної мови, кількість інструментів для автоматичної корекції української мови залишається обмеженою. Наявні рішення здебільшого орієнтовані на стандартні орфографічні та граматичні помилки, тоді як семантична нормалізація діалектних або суржикових конструкцій підтримується рідко і часто реалізується rule-based підходами. Перспективним напрямом є використання нейронних моделей перекладу/редагування, зокрема трансформерних архітектур, які дозволяють навчати систему на паралельних корпусах «нестандартний різновид → стандарт» і масштабувати рішення на різні мовні різновиди [1]. Для коректного порівняння та відтвореної оцінки якості таких систем застосовуються як класичні автоматичні метрики, так і сучасні нейронні фреймворки оцінювання якості перекладу [3].

Метою бакалаврської кваліфікаційної роботи є розробка та експериментальне оцінювання системи автоматичної нормалізації українських діалектів і російсько-українського суржику до літературної української мови на основі нейронних моделей та підходів до роботи з низькоресурсними даними.

Для досягнення поставленої мети необхідно розв'язати такі **задачі**:

- виконати аналіз предметної області та огляд сучасних підходів до

нормалізації діалектів і гібридних мовних форм у задачах NLP;

- проаналізувати доступні джерела даних для задачі «діалект/суржик → літературна українська» та обґрунтувати спосіб формування навчального корпусу;
- розробити та експериментально оцінити baseline-рішення на прикладі одного діалекту (гуцульського), сформувавши відповідний паралельний корпус і навчивши модель нормалізації;
- на основі отриманих результатів масштабувати підхід на кілька діалектів та суржик, сформувавши багатодіалектний корпус і уніфікований пайплайн підготовки даних;
- виконати інструкційне донавчання сучасної україномовної LLM (LaraLLM) для задачі нормалізації та провести експериментальні дослідження якості на контрольних наборах [10];
- реалізувати прикладну систему та продемонструвати її роботу.

Об'єктом дослідження є процеси автоматизованої обробки та нормалізації текстів природної мови українською мовою в умовах високої варіативності мовлення.

Предметом дослідження є методи побудови паралельних корпусів для низькоресурсних мовних варіантів та методи нейронного машинного перекладу/редагування і інструкційного донавчання великих мовних моделей для нормалізації діалектних і суржикових форм [1], [10].

Методами дослідження у роботі є аналіз і підготовка корпусів текстових даних, синтетична генерація паралельних прикладів із використанням лінгвістичних правил та словникових ресурсів, навчання трансформерних моделей для побудови baseline, інструкційне донавчання великих мовних моделей із параметро-ефективними техніками, а також оцінювання якості за автоматичними метриками (BLEU, chrF++, TER) та допоміжними підходами оцінювання якості перекладу [3].

Практична цінність роботи полягає у створенні системи нормалізації, придатної для приведення коротких текстів до літературної української мови. У межах роботи спочатку було побудовано та оцінено baseline-модель нормалізації

для гуцульського діалекту, що дозволило перевірити працездатність підходу та сформувані вимоги до даних і пайплайна навчання. Після цього сформовано багатодіалектний паралельний корпус із підтримкою кількох українських діалектів і російсько-українського суржику, а також виконано спеціалізацію LoraLLM для задачі нормалізації. Для демонстрації роботи системи реалізовано інтерфейс користувача, що дозволяє обирати тип вхідного тексту (діалект/суржик) і отримувати нормалізований результат. Запропонований підхід може бути використаний як окремий інструмент або як компонент більших україномовних NLP-систем.

РОЗДІЛ 1. ЗАГАЛЬНІ ПОЛОЖЕННЯ

1.1. Аналіз літературних джерел

Задача автоматичної нормалізації діалектних форм і суржику до літературної української мови є міждисциплінарною та поєднує підходи щонайменше з двох напрямів: нейронного машинного перекладу (Neural Machine Translation, NMT) і автоматичного редагування/виправлення тексту (Grammatical Error Correction, GEC). У контексті діалектів нормалізацію природно трактувати як “переклад” зі специфічного мовного різновиду на стандарт, тобто як NMT-задачу з низьким рівнем ресурсів. Подібні постановки активно досліджуються для інших мов: зокрема, у роботі [1] показано, що для діалектного перекладу корисною може бути мультизадачність (Multitask Learning), яка допомагає моделі узагальнювати мовні закономірності за дефіциту даних. Для української мови діалектна нормалізація ускладнюється як варіативністю діалектів, так і наявністю суржику — гібридного змішаного мовлення.

Окремим викликом є те, що діалектні та суржикові тексти часто містять нестандартну орфографію, фонетичні відхилення, домішки іншої мови, варіативні морфологічні форми та неуніфіковані скорочення. У таких умовах переваги можуть мати моделі, менш чутливі до помилок токенизації. Наприклад, byte-рівневі підходи, як у ByT5, зменшують залежність від словникових сегментацій і потенційно краще працюють зі “шумними” та нестандартизованими входами [2]. Для багатомовних сценаріїв вагомою є також стратегія збалансованого семплінгу під час переднавчання, яка впливає на якість для низькоресурсних мов; у цьому контексті важливі результати щодо Fair/Effective language sampling, наведені в [17]. Такі підходи створюють теоретичне підґрунтя для вибору архітектур і базових моделей для задачі нормалізації.

Для коректного порівняння моделей та аналізу прогресу потрібне узгоджене оцінювання. У практиці NMT/GEC традиційно використовують BLEU, chrF++, TER та інші автоматичні метрики. Водночас для сучасних

систем перекладу/редагування активно розвиваються нейронні методи оцінювання, зокрема COMET, який дозволяє оцінювати якість ближче до людських суджень і використовуватися як додатковий сигнал для аналізу якості [3]. Також важливим є напрям quality estimation (оцінка якості без еталонного перекладу), який розглядає підходи від “ручних” ознак до великих мовних моделей [7]. Для задачі діалектної нормалізації це актуально, оскільки один і той самий діалектний вираз може мати кілька прийнятних нормалізованих варіантів.

З боку GEC і редагування текстів для української мови сформувались перші суттєві ресурси. Зокрема, UA-GEC — корпус для граматичної корекції та флюентності української мови — є важливим кроком у бік систематичного навчання моделей редагування українською [16]. Іншим прикладом є інструкційно-донавчена модель Spivavtor, орієнтована на задачі україномовного редагування [14]. Оскільки діалектна нормалізація частково перетинається з редагуванням (виправлення ненормативних форм, узгодження, стилістична нейтралізація), ці роботи створюють методологічну основу та надають корисні напрацювання щодо формату даних і способів навчання.

Ключовою проблемою низькоресурсних задач залишається нестача розмічених пар “вхід → еталон”. У цьому контексті практики синтетичного розширення даних є стандартним інструментом. Робота [4] (та її розширення [5]) досліджує моделі, навчені на синтетичних даних для українського GEC, підтверджуючи, що синтетика може суттєво впливати на якість, якщо правильно сконструйовані правила “псування” (errorification) і контроль доменної близькості. Для діалектів і суржику аналогічна логіка застосовується при генерації паралельних пар: або через лінгвістичні правила та словники, або через LLM-керовану генерацію з додатковим контекстом.

Окремої уваги потребує суржик як явище мовного контакту. Лінгвістичні дослідження описують його як змішування українських і російських елементів, що проявляється не лише в лексиці, а й у морфосинтаксисі [9]. З точки зору автоматичної обробки важливим є також виявлення/класифікація змішаного мовлення та його компонентів; цьому присвячені підходи до автоматичного

розпізнавання змішаних мов (на прикладі суржику) [15]. Ці джерела важливі для формалізації типових трансформацій “суржик → норма” та для вибору стратегій генерації даних.

З огляду на зростання ролі decoder-only LLM та інструкційного донавчання, актуальними є роботи про адаптацію сучасних архітектур до української. Зокрема, розглядаються практики донавчання Gemma та Mistral для покращення україномовної репрезентації [6], а також з’являються україномовні або адаптовані під українську моделі, зокрема LapaLLM [10] і MamayLM [11]. Для діалектної нормалізації інструкційний формат (“нормалізуй діалектний текст до літературної української”) є природним, оскільки дозволяє уніфікувати постановку для багатьох діалектів і суржику в межах однієї моделі.

Найбільш релевантним джерелом для побудови практичного рішення в контексті гуцульського діалекту є робота “Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation” [18], де запропоновано підходи до формування паралельних даних і навчання моделей у низькоресурсному діалектному сценарії. Хоча у згаданій роботі напрям перекладу може відрізнитись від задачі нормалізації, запропоновані принципи побудови корпусу, словникова підтримка та підходи до масштабування є методологічно придатними й для постановки “діалект → стандарт”.

Таким чином, аналіз джерел показує: (1) нормалізація діалектів/суржику методично близька до NMT і GEC; (2) ключовими є низькоресурсні практики — синтетичні дані та інструкційне донавчання; (3) для української вже існує базис у вигляді корпусів і моделей редагування, але діалекти та суржик потребують спеціалізованих даних і моделей; (4) сучасні україномовні LLM створюють основу для мультидіалектного рішення.

1.2. Аналіз наборів даних

Для задачі автоматичної нормалізації діалектних форм і російсько-українського суржику до літературної української мови визначальним фактором є якість і репрезентативність даних. На відміну від “класичного” машинного перекладу між мовами, у даній роботі маємо справу з мовними

різними видами однієї мови (або змішаним мовленням), де межа між “помилкою”, “локальною нормою” та “варіантом написання” часто є розмитою. Через це вимоги до корпусу включають не лише достатній обсяг, але й наявність узгоджених еталонів, контроль джерел (ручні/синтетичні), баланс різних типів явищ (лексика, морфологія, синтаксис), а також прозору процедуру формування train/validation/test.

У межах роботи використано комбінований підхід: (1) як опорну базу взято реальні паралельні дані для гуцульського діалекту; (2) для масштабування покриття додано синтетичні пари, згенеровані за участі лінгвістичних правил і словникових ресурсів; (3) для формування “норми” літературної української та забезпечення різноманітності target-сторони використано україномовний датасет редагування/перефразування Spivator (використовується саме як джерело нормативних цільових форм та стилістично природних речень) [14]. Далі наведено детальний аналіз кожного джерела даних та способу їх інтеграції.

1.2.1. Опорні ресурси для гуцульського діалекту: ручний корпус, синтетика та словник

Побудову системи доцільно починати з діалекту, для якого наявні найбільш придатні ресурси. У даній роботі таким “якорем” обрано гуцульський діалект, оскільки для нього доступні (а) вручну анотований паралельний корпус, (б) синтетично згенерований корпус, (в) діалектний словник. Наявність цих компонентів дозволяє повністю відпрацювати процес: від формування датасету й розбиття на підвибірки до навчання моделі та аналізу помилок на реальних прикладах.

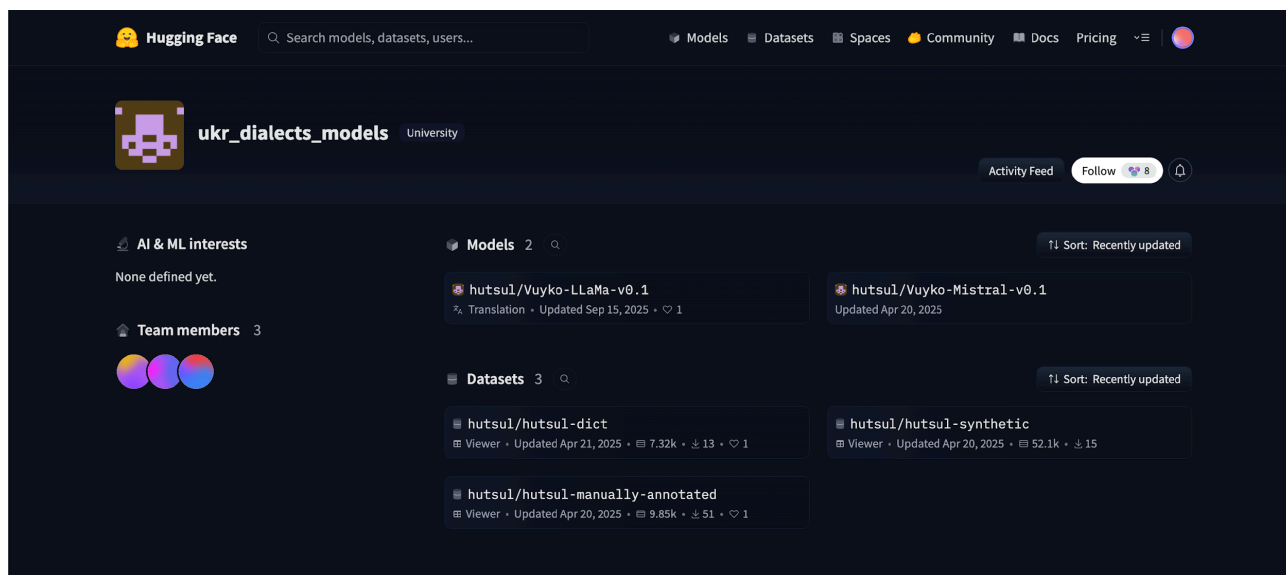


Рис. 1.1. Приклад сторінки з гуцульськими ресурсами на Hugging Face (датасети та моделі, що використовувались як вихідні джерела).

Ручний паралельний корпус.

Вручну анотований корпус є найбільш цінним компонентом, оскільки забезпечує надійність відповідників “діалект → норма”. Саме ці дані рекомендовано використовувати для формування контрольних наборів (validation/test), щоб метрики відображали реальну якість на “справжніх” діалектних прикладах. Для такого корпусу стандартною є структура з двома колонками: source (діалектний текст) та target (літературний відповідник). Важливо підкреслити, що в задачі нормалізації можливі ситуації, коли один діалектний вислів має кілька допустимих літературних варіантів; тому ручна розмітка, як правило, фіксує один узгоджений еталон, а альтернативи можуть розглядатись на етапі якісного аналізу.

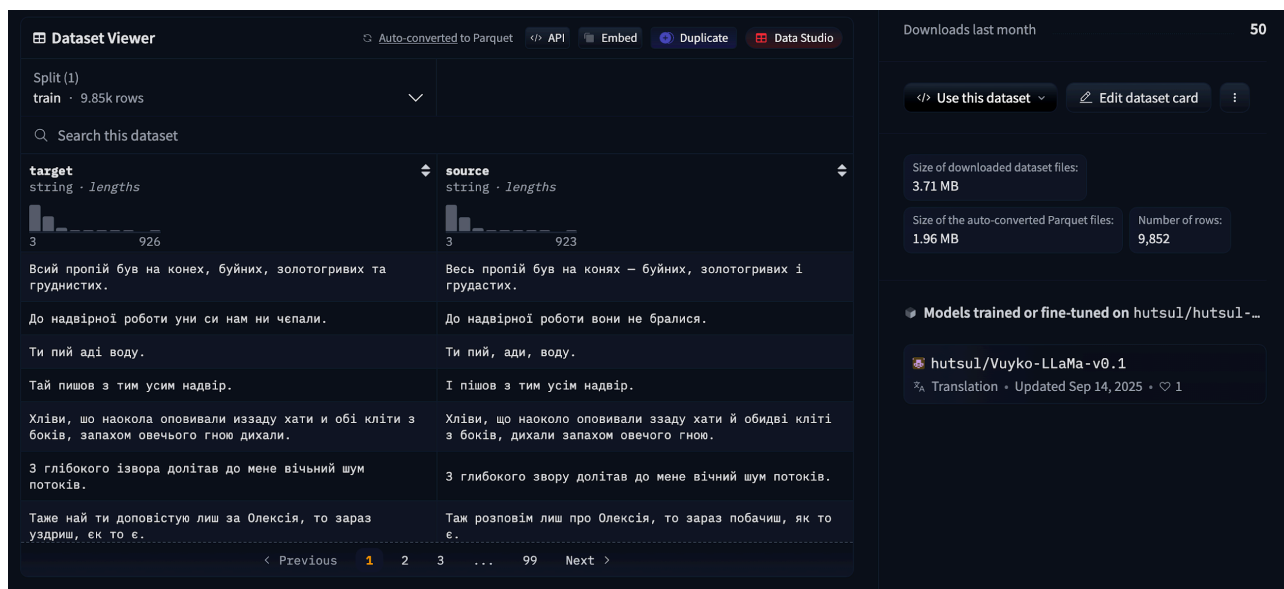


Рис. 1.2. Приклад перегляду вручну анотованого корпусу для гуцульського діалекту (колонки *source/target* та статистика довжин прикладів).

Синтетичний паралельний корпус.

Синтетичний корпус для гуцульського використовується як спосіб збільшення обсягу даних і покриття рідкісних слів/форм. Типова проблема для діалектів полягає в тому, що ручні корпуси часто є “малими” за обсягом, а їх домен може бути вузьким (наприклад, тексти певного жанру). Синтетичні пари дозволяють (а) розширити домен, (б) збільшити частоту специфічної лексики, (в) урізноманітнити структури речень. Разом із тим, синтетика не може замінити ручну розмітку для оцінювання, оскільки містить ризик “штучних” конструкцій і систематичних помилок генератора. Тому синтетичні дані доцільно використовувати переважно у train, а контрольні набори формувати із ручних прикладів.

Діалектний словник.

Словник є ключовим ресурсом, який виконує одразу кілька ролей:

- джерело лексичних відповідників (діалектна форма → літературна);
- “обмежувач” для генерації синтетики (підказує допустимі діалектні варіанти);
- retrieval-база (простий RAG): під час генерації можна підставляти у контекст приклади зі словника, щоб зменшити випадковість і підвищити узгодженість діалектних форм.

Словник також корисний для аналізу помилок: частину помилок моделі можна розкласти на (а) лексичні відповідники, (б) морфологічні/фонетичні трансформації, (в) синтаксичні перебудови. Якщо модель помиляється саме на словникових відповідниках — це ознака проблеми з покриттям або з частотністю відповідних пар у train.

1.2.2. Джерело “норми”: використання Spivavtor як пулу літературної української

Для задачі нормалізації важливо не лише “замінити діалектні слова”, а отримати природний і нормативний текст, що відповідає стилістичним очікуванням. Тому потрібне джерело літературної української з різноманітними конструкціями. У роботі для цього використовувався датасет Spivavtor, який містить пари редагування та перефразування українською мовою, зокрема підзадачі grammatical error correction (gec), paraphrase, simplification тощо [14].

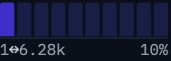
У контексті даної роботи Spivavtor застосовувався не як “діалектний” корпус, а як:

1. **пул цільових (target) речень**, які вже є літературними та стилістично природними;
2. **джерело різноманітних нормативних структур**, корисних для синтетичної генерації пар;
3. **орієнтир для стилю**: цільова сторона нормалізації повинна бути не лише граматично правильною, але й “плинною”.

Datasets: grammarly/spivavtor like 5 Follow Grammarly 200 Dataset card

Split (2)
train · 62.8k rows

Search this dataset

id	src	tgt	task
int64	string · lengths	string · lengths	string · classes
 1e6.28k 10%	 99e176 34.3%	 77e153 29.8%	 gec 44.5%
1	Перефразуйте: Кайс став одержимий нещодавно усвідомленою вразливістю.	Кайс був одержимий своєю нововиявленою вразливістю.	paraphrase
2	Перепишіть речення іншими словами: І вона більше не думатиме про Джонні Сміта та його...	Джонні Сміт і його дивна чарівна посмішка, про яку він ніколи більше не згадав.	paraphrase
3	Виправити граматику: З простими, там, поїсти, поспати не важко.	З простими, там, поїсти, поспати не важко.	gec
4	Виправте всі граматичні помилки: Тоді як ДНК має дві спіралі – інформаційний ряд...	Тоді як ДНК має дві спіралі – інформаційний ряд з'єднаний з комплементарним йому іншим...	gec
5	Зробіть цей текст легше для розуміння: Ви в дуже серйозній біді, містере Штайнфелд.	Містере Штайнфелд, у вас великі проблеми.	simplification
6	Виправте зв'язність в цьому тексті: англiканство вперше прийшло до Бразилії на...	Англiканство вперше прибуло до Бразилії на початку 19 століття, коли португальська...	coherence
7	Покращте зв'язність тексту: пісні у стилі блек-метал часто відрізняються від...	Блек-метал-пісні часто відрізняються від традиційної структури пісень і часто не...	coherence
8	Виправити граматику: Іван побачив сірий берет в гуці на початку Великої Микитської,...	Іван побачив сірий берет у гуці на початку Великої Микитської, але Герцена.	gec

< Previous 1 2 3 ... 628 Next >

Рис. 1.3. Приклад перегляду датасету Spivavtor (колонки src/tgt та мітка типу завдання).

Практична логіка використання Spivavtor як “норми” така: вибирається літературне речення у із Spivavtor (часто з підмножини, де текст є нейтральним за стилем і не надто довгим), після чого генерується діалектний або суржиковий варіант x — за правилами/словником/LLM. Таким чином формується нова пара (x, y) , де y гарантовано належить до літературної української, а x — штучно створений “нестандартний” варіант, який модель вчитиметься нормалізувати.

1.2.3. Генерація синтетичних даних: принципи, сценарії та контроль якості

Оскільки для більшості діалектів і суржику дефіцит ручних пар є критичним, синтетичне розширення є необхідним. В роботі застосовувався підхід “норма → нестандарт”, коли літературний текст виступає еталоном y , а діалектний/суржиковий варіант x генерується. Це має кілька переваг:

- легше отримати багато різноманітних літературних речень (зокрема

з Spivavtor);

- діалектні правила/словники можна застосувати як контрольований механізм “відхилення” від норми;
- формування пар одразу гарантує наявність якісного target.

Умовно можна виділити три сценарії синтетики:

3. Словниково-керована генерація (лексичні заміни + узгодження).

З літературного речення вибираються слова/фрази, для яких є діалектні відповідники у словнику, після чого виконуються заміни та мінімальні морфологічні корекції (закінчення, узгодження, частки). Такий підхід добре покриває лексичний шар, але слабший щодо синтаксичних перебудов.

2. Правилова генерація (фонетика/морфологія/типові конструкції).

Застосовуються регулярні трансформації (наприклад, фонетичні варіанти написання, характерні суфікси, частки). Це підвищує “діалектність” форми, але потребує акуратності, щоб не зруйнувати читабельність або не створити нереалістичні сполучення.

3. LLM-генерація з обмеженнями та RAG-підказками.

LLM генерує діалектний/суржиковий варіант, отримуючи як контекст: (а) інструкцію, (б) приклади діалектних відповідників зі словника, (в) (за потреби) невеликий набір демонстраційних пар з ручного корпусу. Такий режим підвищує природність і різноманітність синтетики, але вимагає суворішого контролю якості.

Контроль якості синтетичних пар — критичний, оскільки “погана” синтетика здатна зіпсувати навчання. У роботі доцільно застосовувати такі евристики/фільтри:

- **фільтрація дублікатів** (однакові x або y);
- **контроль довжин**: надто короткі приклади неінформативні, надто довгі — часто містять помилки або жанровий шум;
- **відсів пар з надто малою/надто великою різницею**:
 - якщо x приблизно рівний y, приклад майже нічого не вчить моделі;
 - якщо відмінність надто велика і змінюється зміст, приклад може

бути некоректним;

- **словникова валідація**: якщо в х присутні “діалектні” слова, перевіряти, чи вони існують у словнику/правилах;
- **вибірковий ручний аудит**: невелика частина синтетики перевіряється вручну для оцінки типових помилок генерації.

1.2.4. Інтеграція джерел у єдиний корпус і організація даних у проєкті

Після збору ручних і синтетичних даних для гуцульського діалекту та підготовки пулу літературних речень (Spivavtor target), корпус інтегрується у єдину структуру, придатну для навчання. Практично це означає:

- уніфікацію формату (source, target);
- додавання метаданих (діалект, тип джерела: manual/synthetic);
- об’єднання файлів у “merged” набір, який використовується як основний train-корпус.

Організація даних у репозиторії важлива з точки зору відтворюваності: по структурі директорій має бути зрозуміло, звідки походять дані, які файли є “сирими”, які — очищеними, і який файл є фінальним для навчання.

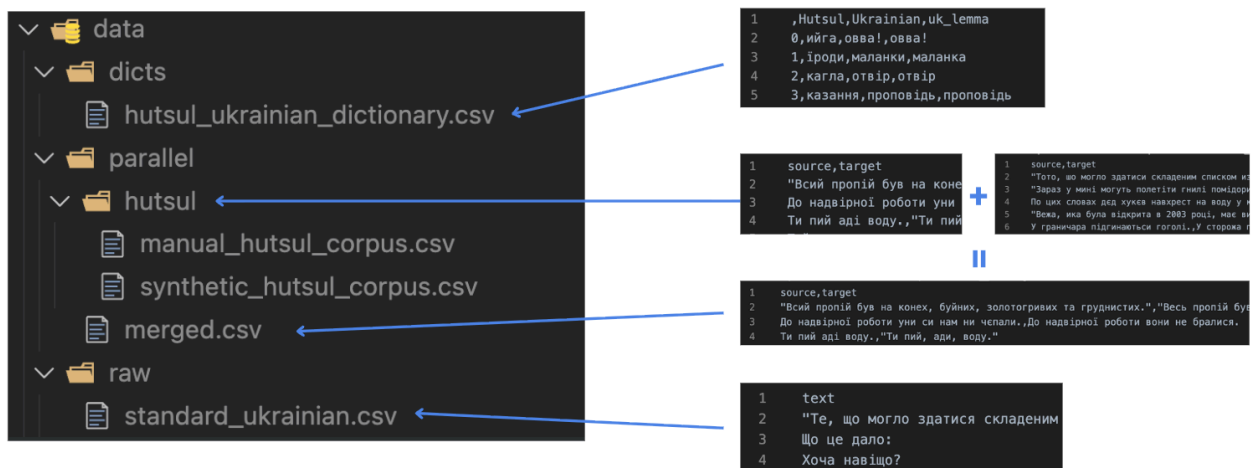


Рис. 1.4. Структура директорій даних у проєкті: словники, ручний корпус, синтетика, merged-корпус і пул літературної української.

Окремо варто зафіксувати ключовий принцип: **контрольні набори (validation/test) формуються з реальних (ручних) пар**, тоді як синтетика використовується переважно в train. Це дозволяє уникнути переоцінки якості на

“штучних” прикладах та отримувати метрики, близькі до реального застосування.

1.2.5. Підготовка вибірок і балансування (для майбутнього мультидіалектного корпусу)

Навіть якщо базовий етап сфокусовано на одному діалекті, при переході до мультидіалектної нормалізації постає проблема дисбалансу: різні діалекти матимуть різний обсяг даних, різний домен (фольклор, побут, сучасні чати), різну відстань до літературної української. Якщо не враховувати це під час навчання, модель може:

- “переадаптуватись” до найбільшого діалекту;
- втратити якість на малих діалектах (catastrophic forgetting);
- генерувати “середній” стиль, який не є достатньо нормативним.

Тому вже на етапі формування даних доцільно закладати:

- **стратифіковане розбиття** за діалектами (і за типом джерела);
- **балансування семплінгу** під час навчання (ваги для діалектів);
- **мінімальні квоти** для рідкісних діалектів у train;
- окремі “діалект-специфічні” контрольні набори для метрик по кожному різновиду.

Ці принципи забезпечують масштабованість пайплайна: додавання нового діалекту стає процесом “додай дані + перезапусти уніфіковану підготовку”, без необхідності переписувати код або змінювати формат.

1.3. Висновки до розділу

У першому розділі проаналізовано літературні джерела та дані, необхідні для побудови системи автоматичної нормалізації діалектів і суржику до літературної української мови. Показано, що задача методично близька до NMT і текстового редагування та потребує нейромережових підходів через високу варіативність діалектних і суржикових форм. Основною практичною складністю є низькоресурсність: для більшості діалектів відсутні достатні паралельні корпуси, тому ключовими стають синтетичне розширення даних і використання зовнішніх ресурсів.

Обґрунтовано вибір гуцульського діалекту як стартового: наявність вручну анотованого корпусу та діалектного словника дозволяє побудувати контрольний baseline-пайплайн і отримати надійну оцінку якості на реальних прикладах. Для формування “норми” літературної української використано датасети редагування (зокрема Spivavtor), що підвищує природність цільових речень і дає опору для генерації синтетики [14]. Визначені принципи даних і підготовки корпусу формують основу для подальшого опису методів навчання, оцінювання та реалізації системи в наступних розділах.

РОЗДІЛ 2. ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

У цьому розділі наведено формалізацію задачі нормалізації діалектних форм та суржику до літературної української, описано використані підходи (послідовність-до-послідовності та decoder-only LLM), обґрунтовано вибір архітектур і режимів навчання, а також подано математичні основи навчання, декодування та оцінювання якості.

4-Phase ML Pipeline:

Phase 1: Data Generation
Input: Standard Ukrainian text
Tool: OpenRouter/Local
Output: Parallel corpus (CSV: source → target)

Phase 2: Model Training
Input: Parallel corpus (data/parallel/merged.csv)
Tool: PyTorch • Transformers
Model: Encoder-Decoder (umT5) or Decoder-Only (Llama/Gemma/Lapa/Mamay)
Output: Fine-tuned checkpoint (models/)

Phase 3: Evaluation
Input: Test set (data/parallel/eval.csv)
Tool: PyTorch • Transformers
Metric: BLEU/chrF++/TER
Output: predictions.txt + references.txt

Phase 4: Inference
Input: Dialect/surzhyk text
Tool: PyTorch • Transformers • Gradio
Output: Normalized Ukrainian text

Рис. 2.1. Узагальнена схема системи: (а) бейзлайн “гуцульський → літературна”, (б) мультидіалектний/суржиковий етап на Lapa LLM, (в) демо-інтерфейс.

2.1. Аналіз методів і підходів до нормалізації діалектів та суржику

2.1.1. Нормалізація як задача перетворення тексту

Нормалізацію діалектних форм та суржику можна розглядати як кероване перетворення тексту, де необхідно зберегти зміст, але змінити форму (лексичну, морфологічну, орфографічну, інколи синтаксичну) відповідно до літературної норми. На відміну від “виправлення граматики” у вузькому сенсі, тут часто присутні **лексичні заміни** (діалектизми), **варіативна орфографія**, **фонетично мотивовані написання**, а також **код-міксинг** (для суржику).

Формально будемо мати вхідний рядок x (діалект/суржик) та бажаний вихідний рядок y (літературна українська). Мета — побудувати модель f_{θ} , що апроксимує відображення:

$$f_{\theta}: x \mapsto y$$

З погляду статистичного моделювання це еквівалентно оцінюванню

умовного розподілу:

$$p_{\theta}(x)$$

Практичні особливості задачі:

- **часткова тотожність**: значна частина токенів у x може збігатися з y ;
- **асиметрія трансформацій**: “виправляти занадто багато” небезпечно (ризик спотворити зміст), але “виправляти занадто мало” — не виконати нормалізацію;
- **багатоваріантність норми**: одна діалектна форма може мати декілька літературних парафразів, особливо для коротких фраз і часток;
- **низькоресурсність**: для більшості діалектів немає великих вручну розмічених паралельних корпусів.

2.1.2. Rule-based підходи та їх обмеження

Rule-based нормалізація опирається на словники відповідників, регулярні правила перетворень і евристики. Її перевага — контрольованість і пояснюваність: якщо відомі правила заміни, можна гарантувати певну поведінку. Однак для українських діалектів і суржику rule-based підхід зазвичай має такі проблеми:

- складно охопити всю лексичну варіативність;
- важко правильно моделювати **контекстні значення** (омонімія/полісемія);
- правила погано працюють із **довгими залежностями** і перебудовами;
- якість деградує при зміні домену (побутові чати, оголошення, студентські конспекти тощо).

Тому rule-based ресурси (словники/правила) у цій роботі розглядаються як допоміжні: (а) для побудови синтетичних даних; (б) для підказок (retrieval) під час генерації синтетики; (в) як інструмент аналізу помилок.

2.1.3. Нейромережеві підходи: seq2seq і decoder-only LLM

У роботі послідовно застосовано дві парадигми.

1. Seq2seq (encoder–decoder) для бейзлайну на одному діалекті (гуцульському).

Переваги:

- стабільне навчання на паралельних даних;
- природна постановка як “переклад” $x \rightarrow y$;
- добре підходить для невеликих/середніх моделей і обмежених ресурсів.

2. Decoder-only LLM (instruction-tuned) для розширення на багато діалектів і суржик.

Переваги:

- універсальність: одна модель може виконувати різні підзадачі (нормалізація різних діалектів, суржик, пояснення правок);
- простіший формат даних через інструкції (prompt $\rightarrow$ response);
- потенційно краща генеративна “плинність” та стилістична якість.

Ключовою базовою моделлю для масштабування обрано **Lara LLM**, оскільки це відкрита українсько-орієнтована LLM, побудована на базі **Gemma-3-12B** та оптимізована під українську за рахунок адаптації токенизатора і прозорого пайплайна тренування [10].

2.2. Постановка задачі та інформаційна модель даних

2.2.1. Формальна постановка

Нехай Σ — алфавіт символів (або множина байтів/токенів), тоді вхідний текст:

$$x = (x_1, x_2, \dots, x_{|x|}), \quad x_i \in \Sigma$$

вихідний текст:

$$y = (y_1, y_2, \dots, y_{|y|}), \quad y_j \in \Sigma$$

Задача — знайти параметри θ , що максимізують правдоподібність тренувальних пар $\{(x^{(k)}, y^{(k)})\}_{k=1}^N$:

$$\theta^* = \arg \arg \sum_{k=1}^N \log \log p_{\theta}!(x^{(k)})$$

У практичній реалізації використовується мінімізація негативного лог-правдоподібності (крос-ентропії):\

$$L(\theta) = - \sum_{k=1}^N \log \log p_{\theta}!(x^{(k)})$$

2.2.2. Подання даних для seq2seq

Для encoder–decoder моделі дані зберігаються у вигляді пар:

- source: діалект/суржик;
- target: літературна українська.

Після токенизації отримуємо послідовності індексів:

$$x = (x_1, ..., x_T), \quad y = (y_1, ..., y_{T'})$$

Модель навчається прогнозувати y_t з урахуванням усіх x та попередніх $y_{<t}$:

$$p_{\theta}(x) = \prod_{t=1}^{T'} p_{\theta}!(x, y_{<t})$$

2.2.3. Подання даних для instruction-tuned LLM (chat format)

Для decoder-only LLM зручно представляти задачу як інструкцію, де модель отримує:

- роль “system” із правилами нормалізації;
- роль “user” із текстом (і, за потреби, міткою діалекту/режиму);
- роль “assistant” із відповіддю-нормою.

Тоді оптимізується:

$$p_{\theta}(\text{system}, \text{user})$$

Приклад структури (концептуально, без прив’язки до конкретних спецтокенів):

- System: “Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого.”

- User: “[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст...”
- Assistant: “...нормалізований текст...”

Цей формат особливо зручний для мультидіалектного сценарію: один і той самий механізм дозволяє перемикати поведінку через підказку (condition) у prompt.

```
* System: "Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого."
* User: "[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст..."
* Assistant: "...нормалізований текст..."
```

Рис. 2.2. Приклад шаблону інструкції для LLM: system/user/assistant та місце для маркера діалекту.

2.3. Математичні основи використаних моделей

2.3.1. Архітектура Transformer: увага та обчислення

Основою і для seq2seq, і для decoder-only підходів є Transformer, який використовує механізм **самоуваги**. Нехай на вході є матриця представлень токенів $H \in R^{T \times d}$. Для одного attention-блоку обчислюються:

$$Q = HW_Q, K = HW_K, V = HW_V,$$

де $W_Q, W_K, W_V \in R^{d \times d_k}$.

Далі:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

Multi-head attention використовує кілька голів і конкатенацію результатів:

$$\text{MHA}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

2.3.2. Seq2seq encoder–decoder: умовна генерація

Encoder перетворює вхід x у контекстні представлення E . Decoder генерує y , використовуючи:

- self-attention по вже згенерованих токенах,
- cross-attention до E .

Тренувальна ціль:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} w_{y_{i,t}} \log \log \left(\hat{y}_{i,t} \right)$$

2.3.3. *Decoder-only LLM: автогресивна модель*

Decoder-only LLM моделює послідовність токенів z (де z включає system+user+assistant у вигляді одного “ланцюжка”):

$$p_{\theta}(z) = \prod_{t=1}^{|z|} p_{\theta}(z_{<t}).$$

На практиці нас цікавить якість саме на токенах відповіді assistant, тому під час навчання можна:

- або рахувати loss по всіх токенах,
- або маскувати loss так, щоб оптимізувати лише частину “assistant”.

2.4. Вибір моделі та обґрунтування: від бейзлайну до Lora LLM

2.4.1. *Бейзлайн для гуцульського діалекту як необхідний етап*

Першим етапом реалізовано бейзлайн лише для гуцульського діалекту. Це методологічно важливо з трьох причин:

1. дозволяє перевірити, що паралельні дані, токенізація та навчання працюють коректно;
2. дає контрольований “полігон” для аналізу помилок (які саме трансформації модель робить/не робить);
3. створює поріг якості, від якого можна вимірювати виграш при переході до LLM та мультидіалектності.

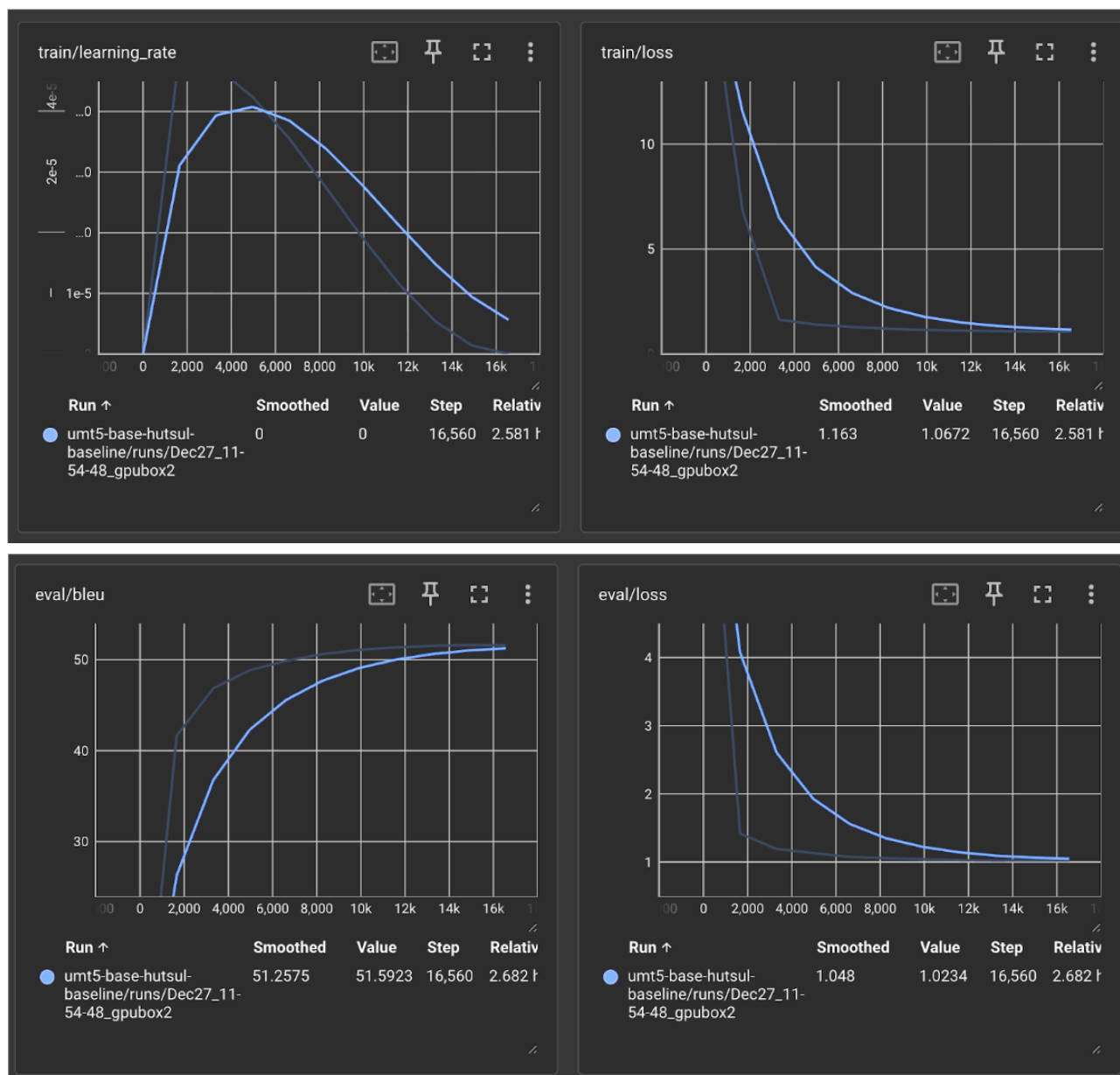


Рис. 2.3. Графік навчання бейзлайну.

2.4.2. Масштабування на мультидіалектність і суржик: вимоги до архітектури

Перехід від одного діалекту до множини діалектів (≈ 13) та суржику ускладнює задачу:

- зростає різноманітність фонетичних і лексичних явищ;
- з'являється потреба у “перемикачі” режимів (який діалект нормалізуємо);
- необхідна краща генеративна здатність для природного стилю.

Decoder-only LLM з інструкційним форматом природно підтримує цю

постановку: можна додавати в prompt маркер діалекту або навіть просити модель самостійно визначити різновид і нормалізувати.

2.4.3. Lora LLM: ключові властивості, важливі для нормалізації

Для мультидіалектного етапу використано **Lora LLM** [10] - відкриту українсько-орієнтовану модель, що базується на **Gemma-3-12B**.

Властивості, які прямо впливають на якість і ефективність нормалізації:

1. Український токенизатор (адаптація словника токенів).

У репозиторії/картці моделі зазначено, що в межах проєкту замінено приблизно 80 тис. токенів із 250 тис. на українські без втрати якості; для української мови це дає скорочення кількості потрібних токенів приблизно у 1.5 рази, а обчислень — суттєво менше для досягнення кращих результатів.

Для задачі нормалізації це важливо, бо діалектні форми і варіативна орфографія можуть “ламати” токенизацію; кращий словник токенів зменшує фрагментацію слів і стабілізує генерацію.

2. Instruction-tuned версія (готовність працювати за інструкціями).

Instruction-режим спрощує інтеграцію в прикладний сервіс: ми формулюємо вимогу до стилю відповіді (лише нормалізований текст, без пояснень) або навпаки — додаємо пояснення за потреби.

3. Довгий контекст.

У картці моделі вказано підтримку довгого контексту (до 128K токенів). Це відкриває шлях до:

- пакетної нормалізації більших документів,
- RAG-підходів (підвантаження словників/правил у контекст),
- додавання прикладів (“in-context learning”) без зміни ваг моделі.

4. Відкритість та відтворюваність.

Команда Lora LLM публікує код і набори даних; у репозиторії декларується висока прозорість пайплайна і доступність для комерційного використання.

Це важливо для дипломної роботи, бо забезпечує відтворюваність і коректне посилання на джерело моделі [10].

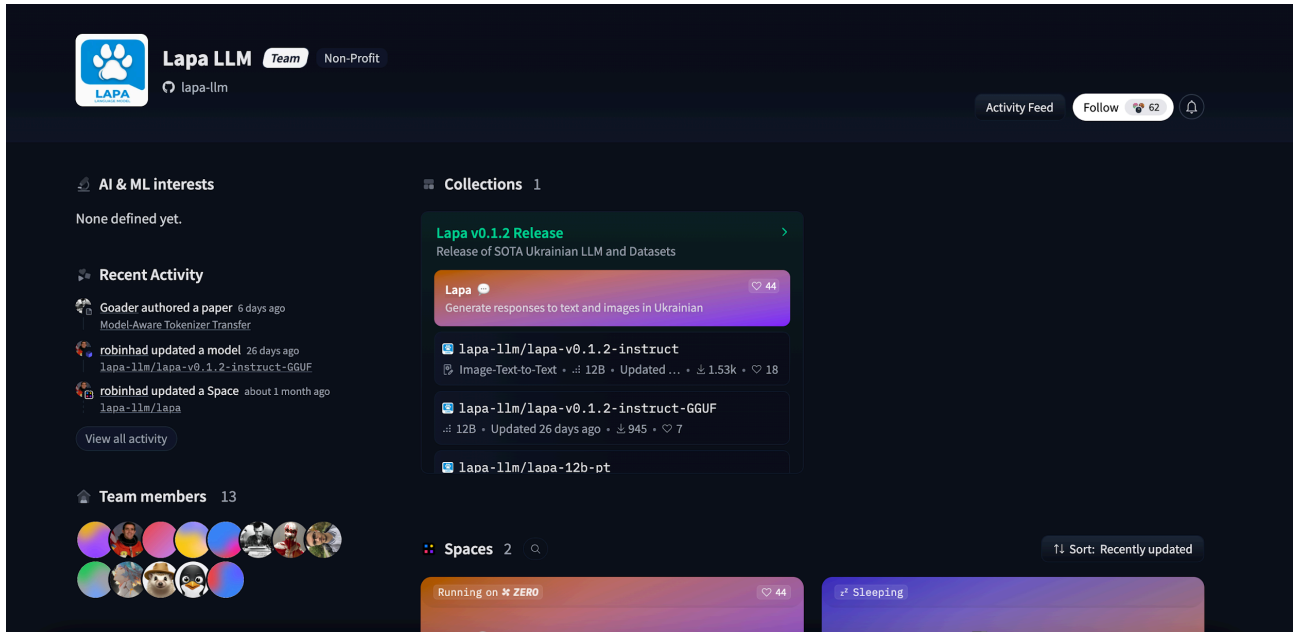


Рис. 2.4. Сторінка моделі Lapa LLM на Hugging Face / GitHub (ключові характеристики: база GPT-3-12B, інструкційна версія, контекст, токенізатор).

2.4.4. Параметро-ефективне донавчання LLM: LoRA/QLoRA

Повне донавчання 12B-моделі є дорогим, тому практично застосовується PEFT (parameter-efficient fine-tuning), найчастіше LoRA.

Нехай у Transformer є матриця ваг $W \in R^{d \times k}$. LoRA додає низькорангову поправку:

$$W' = W + \Delta W, \quad \Delta W = BA,$$

де $B \in R^{d \times r}$, $A \in R^{r \times k}$, $ar \ll (d, k)$.

Тоді кількість тренуваних параметрів зменшується з dk до $r(d+k)$.

У QLoRA (концептуально) базові ваги W зберігаються у квантизованому вигляді W_q , а тренуються лише LoRA-параметри A, B . Можна записати:

$$W_q = \text{Quantize}(W), \quad W' \approx \text{Dequantize}(W_q) + BA.$$

Це дає змогу донавчати великі LLM на обмежених GPU-ресурсах.

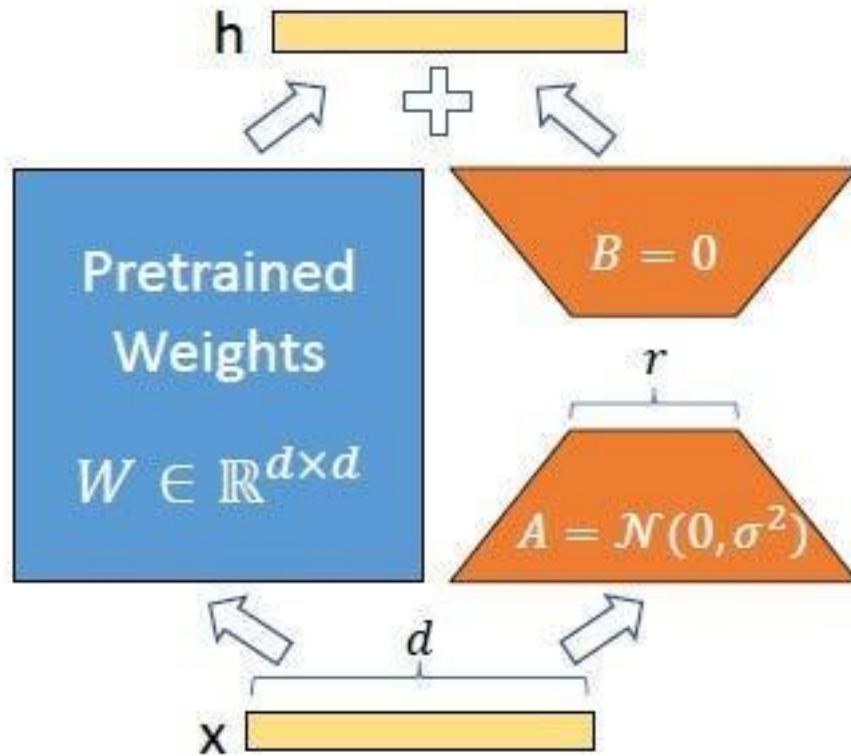


Рис. 2.5. Ілюстрація ідеї LoRA: замість повного оновлення ваг — низькорангова поправка в attention/FFN шарах.

2.5. Метрики якості та цільові функції навчання

2.5.1. Крос-ентропія та маскування втрат

Стандартна функція втрат для генеративних моделей:

$$L = - \sum_{t=1}^{T'} \log \log p_{\theta}(x, y_{<t}).$$

Для instruction-tuned LLM доцільно оптимізувати лише токени відповіді assistant. Нехай $m_t \in \{0, 1\}$ — маска (1 лише на позиціях відповіді), тоді:

$$L_{mse} = - \sum_{t=1}^T m_t \log \log p_{\theta}(z_{<t}).$$

Це зменшує ризик “перевчити” модель відтворювати системні інструкції і збільшує фокус саме на якості нормалізації.

2.5.2. Декодування: greedy та beam search

Під час інференсу потрібно знайти:

$$\hat{y} = \arg \arg \log \log p_{\theta}(x) .$$

Greedy-декодування:

$$\hat{y}_t = \arg \arg p_{\theta}(x, \hat{y}_{<t}) .$$

Beam search підтримує В найкращих часткових гіпотез. Часто застосовують штраф за довжину (length penalty):

$$s(y) = \frac{1}{|y|^{\alpha}} \sum_{t=1}^{|y|} \log \log p_{\theta}(x, y_{<t}) ,$$

де α — параметр (наприклад, 0.6 ... 1.0).

Для нормалізації коротких текстів beam search може підвищувати стабільність, але інколи стимулює “надмірні правки”, тому параметри декодування підбирають експериментально.

2.5.3. BLEU та chrF++ для нормалізації

Оскільки задача близька до перекладу, застосовні метрики MT. BLEU базується на модифікованій n-грамній точності та штрафі за короткість:

$$BLEU = BP \cdot \exp \exp \left(\sum_{n=1}^N w_n \log \log p_n \right) ,$$

де p_n — модифікована точність n-грам, w_n — ваги (часто $w_n = \frac{1}{N}$), а BP (brevity penalty):

$$BP = \begin{cases} 1, & \text{if } c > r \\ \exp(1 - r/c), & \text{if } c \leq r \end{cases}$$

де c — довжина гіпотези, r — довжина еталону.

Для мов із багатою морфологією символні метрики часто корисніші. chrF/chrF++ вимірює F-міру на символних n-грамах (chrF++) також включає інформацію про слова (word n-grams). Узагальнено:

$$F_{\beta} = \frac{(1+\beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} ,$$

де P — точність, R — повнота для n-грам (символьних/словних), β задає вагу повноти.

2.5.4. Семантичні метрики та оцінювання якості перекладу/нормалізації

Для нормалізації важливо не лише буквальне співпадіння, а і **збереження змісту**. Тому доцільно застосовувати нейромережеві метрики, наприклад COMET [3], які корелюють з людськими оцінками якості перекладу/редагування.

Концептуально COMET оцінює трійку (source, hypothesis, reference) і повертає скаляр якості:

$$q = \text{COMET}(x, \hat{y}, y),$$

де q зазвичай нормований (залежить від конкретної версії моделі). Детальна архітектура і навчання COMET виходять за межі цього підрозділу, але важливо, що це приклад метрики, яка краще відбиває семантичну коректність, ніж лише n -грамні збіги [3].

2.6. Інформаційне забезпечення навчання: баланс, семплінг, контроль доменів

2.6.1. Дисбаланс діалектів та принципи семплінгу

У мультидіалектному корпусі обсяги різних діалектів майже неминуче різні. Якщо навчати модель на “сирому” об’єднанні, то оптимізація зміститься до найбільшого діалекту. Один із способів — вводити ваги семплінгу:

$$P(\text{dialect} = i) \propto n_i,$$

де n_i — кількість прикладів для діалекту i , а $\gamma \in [0, 1]$ керує вирівнюванням:

$\gamma = 1$: пропорційно розмірам (без вирівнювання),

$\gamma = 0$: рівномірно по діалектах.

Або можна робити “квоти” на епоху: однакову кількість батчів на кожен діалект.

2.6.2. Мультизасковок формат і маркери режимів

Для LLM зручно навчати один “універсальний” формат:

- нормалізація діалекту (із маркером діалекту),
- нормалізація суржику (із маркером “SURZHUK”),

- за потреби — варіант “нормалізуй і коротко поясни правки” (окремий режим).

Тоді система фактично розв’язує множину умовних задач:

$$p_{\theta}(x, t),$$

де t — тип/режим (діалект/суржик/пояснення). Це зменшує потребу в окремих спеціалізованих моделях.

2.7. Висновки до розділу

У розділі наведено математичну постановку задачі нормалізації як умовної генерації $p_{\theta}(x)$, описано два послідовні етапи підходу: бейзлайн encoder–decoder для одного діалекту та масштабування через instruction-tuned decoder-only LLM. Подано ключові формули Transformer-уваги, цільових функцій (крос-ентропія, маскування для assistant), принципів декодування (greedy/beam search) та метрик оцінювання (BLEU, chrF++, нейромережеві метрики). Окремо обґрунтовано вибір Lora LLM як українсько-орієнтованої бази на Gemma-3-12B із адаптованим токенизатором, довгим контекстом та відкритим пайплайном [10].

РОЗДІЛ 3. ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ

3.1. Засоби реалізації.

Для розв'язання поставленої задачі, а саме розробки алгоритму та системи для його використання було використано мову програмування Python. Сукупність всіх алгоритмів машинного навчання та математичного апарату, який потрібен для реалізації нейронних мереж не є частиною якоїсь мови програмування.

3.2. Вимоги до технічного та програмного забезпечення.

Для виконання поставленого завдання буде використовуватись Python без хмарного середовища. При розробці нейронної мережі тренування мережі буде відбуватись на CPU.

3.3. Висновки до розділу

В даному розділі було проведено огляд можливих середовищ розробки для вирішення задачі розпізнавання рукописного тексту.

ЗАГАЛЬНІ ВИСНОВКИ

В ході виконання даної бакалаврської кваліфікаційної роботи по розпізнаванню рукописного тексту було ...

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. L. H. Baniata, S. Park, i S.-B. Park, «A Neural Machine Translation Model for Arabic Dialects That Utilizes Multitask Learning (MTL)», *Computational Intelligence and Neuroscience*, вип. 2018, с. 1–10, Груд 2018, doi: [10.1155/2018/7534712](https://doi.org/10.1155/2018/7534712).
2. L. Xue *et al.*, «ByT5: Towards a token-free future with pre-trained byte-to-byte models», 08, Березень 2022, *arXiv*: arXiv:2105.13626. doi: [10.48550/arXiv.2105.13626](https://doi.org/10.48550/arXiv.2105.13626).
3. R. Rei, C. Stewart, A. C. Farinha, i A. Lavie, «COMET: A Neural Framework for MT Evaluation», 19, ЖОВТЕНЬ 2020, *arXiv*: arXiv:2009.09025. doi: [10.48550/arXiv.2009.09025](https://doi.org/10.48550/arXiv.2009.09025).
4. M. Bondarenko, A. Yushko, A. Shportko, i A. Fedorych, «Comparative Study of Models Trained on Synthetic Data for Ukrainian Grammatical Error Correction».
5. M. Bondarenko, A. Yushko, A. Shportko, i A. Fedorych, «Comparative Study of Models Trained on Synthetic Data for Ukrainian Grammatical Error Correction».
6. A. Kiulian *et al.*, «From Bytes to Borsch: Fine-Tuning Gemma and Mistral for the Ukrainian Language Representation», 14, КВІТЕНЬ 2024, *arXiv*: arXiv:2404.09138. doi: [10.48550/arXiv.2404.09138](https://doi.org/10.48550/arXiv.2404.09138).
7. H. Zhao *et al.*, «From Handcrafted Features to LLMs: A Brief Survey for Machine Translation Quality Estimation», в 2024 *International Joint Conference on Neural Networks (IJCNN)*, Чер 2024, с. 1–10. doi: [10.1109/IJCNN60899.2024.10650457](https://doi.org/10.1109/IJCNN60899.2024.10650457).
8. R. Kovalchuk, M. Romanyshyn, i P. Ivaniuk, «Introducing OmniGEC: A Silver Multilingual Dataset for Grammatical Error Correction».
9. K. Kent, «Language Contact: Morphosyntactic Analysis of Surzhyk Spoken in Central Ukraine», *Language Contact*.
10. «LapaLLM». [Online]. Доступний у: <https://huggingface.co/lapa-llm>
11. «MamayLM». [Online]. Доступний у:

12. R. Fedchuk i V. Vysotska, «Mathematical model of a decision support system for identification and correction of errors in Ukrainian texts based on machine learning».
13. M. Haliuk, «On the Path to Make Ukrainian a High-Resource Language».
14. A. Saini, A. Chernodub, V. Raheja, i V. Kulkarni, «Spivavtor: An Instruction Tuned Ukrainian Text Editing Model», 29, Квітень 2024, *arXiv*: arXiv:2404.18880. doi: [10.48550/arXiv.2404.18880](https://doi.org/10.48550/arXiv.2404.18880).
15. N. Sira, G. M. D. Nunzio, i V. Nosilia, «Towards an Automatic Recognition of Mixed Languages in R: The Case of Ukrainian-Russian Hybrid Language Surzhyk».
16. O. Syvokon, O. Nahorna, P. Kuchmiichuk, i N. Osidach, «UA-GEC: Grammatical Error Correction and Fluency Corpus for the Ukrainian Language», в *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, Dubrovnik, Croatia: Association for Computational Linguistics, 2023, с. 96–102. doi: [10.18653/v1/2023.unlp-1.12](https://doi.org/10.18653/v1/2023.unlp-1.12).
17. H. W. Chung *et al.*, «UniMax: Fairer and more Effective Language Sampling for Large-Scale Multilingual Pretraining», 18, Квітень 2023, *arXiv*: arXiv:2304.09151. doi: [10.48550/arXiv.2304.09151](https://doi.org/10.48550/arXiv.2304.09151).
18. R. Kyslyi, Y. Maksymiuk, i I. Pysmennyi, «Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation», 09, Червень 2025, *arXiv*: arXiv:2506.07617. doi: [10.48550/arXiv.2506.07617](https://doi.org/10.48550/arXiv.2506.07617).

Додаток А.

Додаток Б.